SUPERTEAM ZEROCLAW BOUNTY

ZeroClaw Solana Payment Desk

Submission Write-up

ZeroClaw Solana Payment Desk

One-line summary: a ZeroClaw agent workflow for Solana payments where the agent can request, build, and confirm a payment, but cannot custody funds, sign transactions, or silently change payment terms.

Primary code links:

- Plugin repo, pinned commit: https://github.com/Fianko-codes/zeroclaw-plugins/tree/2c3592afa241603bd34311ba392e7214e9ef0a41
- Shared Solana core, pinned commit: https://github.com/Fianko-codes/zeroclaw-solana/tree/09d73652be97a8f348938feac4b022cb049b0c35
- Reproduction runner: https://github.com/Fianko-codes/zeroclaw-solana/blob/09d73652be97a8f348938feac4b022cb049b0c35/demo/reproduce.sh
- Security audit / threat review: https://github.com/Fianko-codes/zeroclaw-solana/blob/09d73652be97a8f348938feac4b022cb049b0c35/M5_SECURITY_AUDIT.md

Submission/supporting links after `submission/` is pushed to GitHub:

- Operator SOP: https://github.com/Fianko-codes/zeroclaw-solana/blob/agent/m5-payment-confirm/submission/supporting/OPERATOR_SOP.md
- Agent operating skill: https://github.com/Fianko-codes/zeroclaw-solana/blob/agent/m5-payment-confirm/submission/supporting/AGENT_SKILL.md
- Redacted config template: https://github.com/Fianko-codes/zeroclaw-solana/blob/agent/m5-payment-confirm/submission/supporting/redacted-config.toml
- Evidence index: https://github.com/Fianko-codes/zeroclaw-solana/blob/agent/m5-payment-confirm/submission/supporting/EVIDENCE_INDEX.md
- Video script: https://github.com/Fianko-codes/zeroclaw-solana/blob/agent/m5-payment-confirm/submission/video/VIDEO_SCRIPT.md
- Last-minute recording runbook: https://github.com/Fianko-codes/zeroclaw-solana/blob/agent/m5-payment-confirm/submission/video/LAST_MINUTE_RECORDING_RUNBOOK.md
- One-pager PDF: https://github.com/Fianko-codes/zeroclaw-solana/blob/agent/m5-payment-confirm/submission/pdf/ZeroClaw_One_Pager.pdf
- Supporting packet PDF: https://github.com/Fianko-codes/zeroclaw-solana/blob/agent/m5-payment-confirm/submission/pdf/ZeroClaw_Submission_Packet.pdf

What it does

ZeroClaw Solana Payment Desk lets an operator run a narrow payment workflow from a real agent channel:

1. **Request** — `solana_pay_request` turns invoice terms into a Solana Pay URL, QR payload, concise approval text, and deterministic payment reference.

2. **Build** — `spl_transfer_build` prepares exactly one allowlisted SPL-token transfer as an unsigned transaction proposal. It simulates the proposal and derives the human approval summary from the final serialized transaction bytes.
3. **Confirm** — `solana_pay_confirm` re-derives the payment reference from the original invoice terms, reads candidate transactions, decodes their raw bytes locally, and returns `paid: true` only when the recipient token account balance increased by the exact requested amount.

The key design point: the agent never receives wallet secrets and never signs or submits a transaction. It can prepare and verify a payment workflow; the wallet owner still approves in Phantom or another external signer.

The request and confirm tools are bound by the same invoice fields: recipient, mint, amount, and invoice ID. If any of those fields change, the payment reference changes. That prevents the model from confirming an unrelated transfer as paid.

Who it is for

This is for:

- Small merchants accepting a narrow set of SPL-token payments.
- Operators who want an agent-native payment desk without giving the agent custody.
- Teams that need reproducible, policy-bound Solana payment tools instead of a general wallet bot.
- Hackathon/demo operators who want terminal + phone proof that an agent can coordinate a real Solana payment flow safely.

This is not a trading bot, custody wallet, autonomous settlement engine, payment watcher, or general-purpose Solana executor.

ZeroClaw features used

ZeroClaw feature	How this submission uses it
WASM tool plugins / WIT v0	Three self-contained `wasm32-wasip2` components with thin WIT bindings and a small shared Rust core.
Per-plugin permissions	Request uses config only. Build/confirm use config plus HTTP RPC. No filesystem, shell, process, socket, environment, or secret-store access.
Operator-owned config	ZeroClaw injects plugin config and strips caller-supplied `__config`. Recipient allowlists, mint allowlists, caps, sender, RPC URL, and nonce mode are operator policy.
Human approval boundary	The build tool returns an unsigned proposal. Signing and submission stay outside ZeroClaw in the user's wallet.
Real channel compatibility	The demo flow is channel-agnostic and can be shown through Telegram, terminal, and phone mirroring.

ZeroClaw feature	How this submission uses it
Installable plugin packaging	Each plugin has a manifest and WASM component build path compatible with ZeroClaw's plugin packaging model.

What I built

I built three ZeroClaw Solana payment plugins plus a shared low-level Solana core:

- `solana-pay-request`: generates a Solana Pay transfer URL and deterministic reference without network access.
- `spl-transfer-build`: creates a constrained unsigned SPL transfer proposal, includes idempotent ATA creation, simulates it, decodes the final bytes, and refuses unsupported transaction shapes.
- `solana-pay-confirm`: verifies a candidate payment by checking the reference account, recipient ATA, decoded transaction, and recipient balance delta.
- `nanosol`: small shared Solana primitives for public keys, decimal parsing, ATAs/PDAs, transfer encoding, message inspection, and narrow JSON-RPC parsing.

Direct plugin links:

- `solana-pay-request` README: https://github.com/Fianko-codes/zeroclaw-plugins/blob/2c3592afa241603bd34311ba392e7214e9ef0a41/plugins/solana-pay-request/README.md
- `solana-pay-request` evidence: https://github.com/Fianko-codes/zeroclaw-plugins/blob/2c3592afa241603bd34311ba392e7214e9ef0a41/plugins/solana-pay-request/EVIDENCE.md
- `solana-pay-request` manifest: https://github.com/Fianko-codes/zeroclaw-plugins/blob/2c3592afa241603bd34311ba392e7214e9ef0a41/plugins/solana-pay-request/manifest.toml
- `spl-transfer-build` README: https://github.com/Fianko-codes/zeroclaw-plugins/blob/2c3592afa241603bd34311ba392e7214e9ef0a41/plugins/spl-transfer-build/README.md
- `spl-transfer-build` evidence: https://github.com/Fianko-codes/zeroclaw-plugins/blob/2c3592afa241603bd34311ba392e7214e9ef0a41/plugins/spl-transfer-build/EVIDENCE.md
- `spl-transfer-build` manifest: https://github.com/Fianko-codes/zeroclaw-plugins/blob/2c3592afa241603bd34311ba392e7214e9ef0a41/plugins/spl-transfer-build/manifest.toml
- `solana-pay-confirm` README: https://github.com/Fianko-codes/zeroclaw-plugins/blob/2c3592afa241603bd34311ba392e7214e9ef0a41/plugins/solana-pay-confirm/README.md
- `solana-pay-confirm` evidence: https://github.com/Fianko-codes/zeroclaw-plugins/blob/2c3592afa241603bd34311ba392e7214e9ef0a41/plugins/solana-pay-confirm/EVIDENCE.md
- `solana-pay-confirm` manifest: https://github.com/Fianko-codes/zeroclaw-plugins/blob/2c3592afa241603bd34311ba392e7214e9ef0a41/plugins/solana-pay-confirm/manifest.toml
- Shared `nanosol` README: https://github.com/Fianko-codes/zeroclaw-solana/blob/09d73652be97a8f348938feac4b022cb049b0c35/nanosol/README.md
- Shared `nanosol` source: https://github.com/Fianko-codes/zeroclaw-solana/blob/09d73652be97a8f348938feac4b022cb049b0c35/nanosol/src/lib.rs

Custody tier

Component	Tier	Can do	Cannot do
`solana-pay-request`	T1 request tool	Construct a payment request and reference	Read chain state, sign, submit, or access keys
`spl-transfer-build`	T1 build tool	Read allowlisted chain state and return one simulated unsigned transfer	Sign, submit, store keys, issue arbitrary instructions, or build multi-recipient transfers
`solana-pay-confirm`	T0 read-only verifier	Read and verify candidate payment transactions	Construct, sign, submit, or move funds

Overall custody tier: **non-custodial agent workflow**. ZeroClaw coordinates payment work; custody remains with the external wallet signer.

Threat model

Untrusted inputs:

- Model instructions.
- User chat messages.
- Invoice text.
- Tool arguments.
- RPC responses.
- Candidate transaction signatures.

Controls:

- Unknown fields are rejected.
- Amounts use strict decimal parsing and integer units, not floating point.
- Recipient, mint, amount cap, sender, RPC endpoint, and durable-nonce mode come from operator config, not from the model.
- Caller-supplied `__config` is stripped by the host boundary.
- The transfer builder simulates before returning.
- The approval summary is generated only after decoding the final serialized transaction bytes.
- The confirmation tool re-derives the invoice reference and checks raw transaction bytes plus recipient balance delta.
- No private key, seed phrase, signing request, or wallet secret belongs in prompts, config, tool inputs, repo files, environment variables, or the video.

Known limits:

- The signer still must inspect and approve the transaction in the external wallet.
- A single malicious RPC can lie about chain state. The confirmation design supports independently operated RPC endpoints, but this demo should be treated as a bounded operator workflow, not final production settlement infrastructure.

- Components are stateless. Daily limits, duplicate-notification suppression, and long-running polling belong in the operator SOP or surrounding system.
- Recent-blockhash proposals can expire. Durable-nonce mode exists for longer approval queues and has explicit nonce-consumption warnings.

How another operator can reproduce it

Use the pinned repos above, install Rust `1.96.1` and the `wasm32-wasip2` target, then run:

```
./demo/reproduce.sh
```

For a faster smoke run:

```
./demo/reproduce.sh --fast
```

The exact operator steps are documented here:

- Operator SOP: https://github.com/Fianko-codes/zeroclaw-solana/blob/agent/m5-payment-confirm/submission/supporting/OPERATOR_SOP.md
- Agent operating skill: https://github.com/Fianko-codes/zeroclaw-solana/blob/agent/m5-payment-confirm/submission/supporting/AGENT_SKILL.md
- Redacted config: https://github.com/Fianko-codes/zeroclaw-solana/blob/agent/m5-payment-confirm/submission/supporting/redacted-config.toml
- Evidence index: https://github.com/Fianko-codes/zeroclaw-solana/blob/agent/m5-payment-confirm/submission/supporting/EVIDENCE_INDEX.md

Evidence and reliability

Evidence included:

- Locked Rust tests for the shared core and plugins.
- Host-side tests and WASM-side checks.
- Golden vectors against official Solana interfaces.
- Adversarial tests for forged config, recipient/mint/cap swaps, prompt-injection text, malformed RPC envelopes, and transaction-byte mutations.
- Real ZeroClaw-host execution records for the request/build/confirm components.
- Real devnet externally signed acceptance transaction for the transfer builder.
- Mainnet read-only confirmation behavior for the verifier.

Evidence links:

- Evidence index: https://github.com/Fianko-codes/zeroclaw-solana/blob/agent/m5-payment-confirm/submission/supporting/EVIDENCE_INDEX.md
- Security audit: https://github.com/Fianko-codes/zeroclaw-solana/blob/09d73652be97a8f348938feac4b022cb049b0c35/M5_SECURITY_AUDIT.md
- Upstream ZeroClaw plugin PR: https://github.com/zeroclaw-labs/zeroclaw-plugins/pull/54

- Solana Pay transfer request spec: https://github.com/solana-foundation/solana-pay/blob/master/SPEC.md#specification-transfer-request

Honest limit: the evidence does not claim a first-party recorded `paid: true` request-to-payment-to-confirm loop unless the demo video shows that exact disposable devnet loop. The design supports it, and the recording SOP is written to capture it, but the submission keeps tested evidence separate from intended behavior.

What I am submitting in Discord

- Demo video: real agent, real Telegram channel, terminal, and mirrored phone; no slides.
- Write-up: this document.
- One-pager PDF: `submission/pdf/ZeroClaw_One_Pager.pdf`
- Supporting packet PDF: `submission/pdf/ZeroClaw_Submission_Packet.pdf`
- Public code links: pinned plugin repo and pinned shared Solana core repo above.

Files that must be uploaded or pushed

To make every link above work, push these local files/folders to the public `Fianko-codes/zeroclaw-solana` repo:

- `submission/WRITEUP.md`
- `submission/ONE_PAGER.md`
- `submission/supporting/OPERATOR_SOP.md`
- `submission/supporting/AGENT_SKILL.md`
- `submission/supporting/redacted-config.toml`
- `submission/supporting/EVIDENCE_INDEX.md`
- `submission/video/VIDEO_SCRIPT.md`
- `submission/video/LAST_MINUTE_RECORDING_RUNBOOK.md`
- `submission/pdf/ZeroClaw_One_Pager.pdf`
- `submission/pdf/ZeroClaw_Submission_Packet.pdf`

For Discord itself, upload or link:

- The video.
- The one-pager PDF.
- The supporting packet PDF.
- The public GitHub repo links.

Do not upload secrets, wallet private keys, seed phrases, real customer data, raw keypair JSON files, or unredacted RPC/API credentials.